

Informe de proyecto: Sistema web de materiales

Alacena & Molde

Fecha de elaboración: 04/10/2026

Integrantes registrados: Pablo Gutiérrez, Matías Gallardo y Álvaro García.

Repositorio: github.com/41e23/catalogo-reposter-a | Rama de entrega: entrega-materiales.

Problemática, continuidad y entidad

El proyecto continúa el catálogo de un emprendimiento de repostería. La entidad Material permite registrar insumos y controlar nombre, categoría, stock, precio y descripción. El CRUD se implementa en la aplicación materiales y se relaciona con el catálogo y la consulta de proveedores.

La estructura contiene cuatro aplicaciones Django: catalogo, inicio, materiales y proveedores. Los usuarios responsables del inventario pueden consultar y modificar materiales desde la interfaz web; el personal autorizado puede administrarlos desde Django Admin.

Base de datos, entorno y estado de verificación

El proyecto admite PostgreSQL mediante DB_ENGINE y las variables DB_NAME, DB_USER, DB_PASSWORD, DB_HOST y DB_PORT cargadas desde .env. El archivo .env.example contiene solo valores de muestra y .env está excluido por .gitignore. psycopg está declarado en requirements.txt.

Estado: en el entorno usado para esta revisión no se detectó un servidor PostgreSQL local. Las pruebas ejecutadas aquí usaron SQLite como respaldo; por tanto, la conexión real a PostgreSQL, la verificación en pgAdmin y sus capturas siguen pendientes de realizar en un equipo con PostgreSQL configurado.

Modelo, migraciones, datos de ejemplo y Admin

Material está definido en materiales/models.py y su migración inicial está en materiales/migrations/0001_initial.py. El modelo está registrado en Django Admin con columnas, filtro por categoría y búsqueda. Una prueba automatizada inicia sesión como superusuario de prueba y verifica que el listado administrativo muestre materiales.

La fixture materiales_demo carga tres registros de ejemplo mediante Django. Para completar la demostración, el equipo debe crear un superusuario local con createsuperuser, iniciar sesión en /admin/, modificar un registro y eliminar otro; no se guardan credenciales en este repositorio.

materiales/admin.py

```

materiales/admin.py
from django.contrib import admin

from .models import Material

@admin.register(Material)
class MaterialAdmin(admin.ModelAdmin):
    list_display = ("nombre", "categoria", "stock", "precio")
    list_filter = ("categoria",)
    search_fields = ("nombre", "categoria", "descripcion")

```

config/urls.py

```

config/urls.py
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("catalogo.urls")),
    path("inicio/", include("inicio.urls")),
    path("materiales/", include("materiales.urls")),
    path("proveedores/", include("proveedores.urls")),
]

```

materiales/migrations/0001_initial.py

```

materiales/migrations/0001_initial.py
# Generated by Django 5.2.17 on 2026-10-04 19:17

from django.db import migrations, models

class Migration(migrations.Migration):

    initial = True

    dependencies = [
    ]

    operations = [
        migrations.CreateModel(
            name='Material',
            fields=[
                ('id', models.BigAutoField(auto_created=True, primary_key=True, serialize=False, verbose_name='ID')),
                ('nombre', models.CharField(max_length=100)),
                ('categoria', models.CharField(max_length=50)),
                ('stock', models.PositiveIntegerField()),
                ('precio', models.DecimalField(decimal_places=2, max_digits=10)),
                ('descripcion', models.TextField(blank=True)),
            ],
        ),
    ]

```

Captura de código y estructura del proyecto

A continuación se muestran bloques de código reales del proyecto, presentados con un estilo visual tipo captura de editor de texto, para facilitar la explicación técnica del CRUD de materiales y la estructura del sistema.

1. Introducción

Este proyecto tuvo como objetivo desarrollar una aplicación web con Django para gestionar materiales de una repostería. Se implementó un CRUD funcional que permite visualizar, registrar, actualizar y eliminar materiales, manteniendo una estructura organizada y con validaciones necesarias para evitar errores en la entrada de datos.

2. Objetivos

- Listar materiales mediante consultas ORM de Django.
- Crear nuevos registros desde un formulario web.
- Editar materiales ya existentes.
- Eliminar materiales con confirmación.
- Mantener validación de campos, control de errores 404 y protección CSRF.

3. Requisitos funcionales

La aplicación debía incorporar CRUD completo, uso de `get_object_or_404()`, manejo adecuado de formularios, rutas específicas para cada operación y una vista de listado con acceso directo a cada acción. Además, se contempló validación para campos vacíos y valores inválidos.

3.1 Lista específica del CRUD solicitado

En `materiales/views.py` se reemplazaron los datos escritos manualmente por consultas ORM y se implementaron las funciones para listar, crear, editar y eliminar materiales. Las funciones de edición y eliminación usan `get_object_or_404()` para registros inexistentes. `materiales/urls.py` define las rutas de listado, creación, edición y eliminación. La plantilla `lista.html` muestra los materiales y sus acciones; `formulario.html` sirve para crear y editar; `eliminar.html` confirma la acción y permite cancelarla. Los formularios incluyen `{% csrf_token %}` y la eliminación solo modifica datos mediante POST. Las pruebas comprueban listado, creación válida, campos vacíos, dato inválido, edición, cancelación, confirmación, 404 y CSRF. Las capturas de cada flujo aparecen en la sección 7.

4. Desarrollo del sistema

El módulo de materiales se estructura con una app Django llamada `materiales`. Este módulo incluye un modelo que almacena los datos, un formulario de validación, las vistas de lógica y una serie de templates para mostrar la interfaz. El patrón seguido fue el típico de Django MVC, aprovechando el ORM para gestionar todas las operaciones contra la base de datos.

4.1 Modelo de datos

`materiales/models.py`

```
materiales/models.py
from django.db import models

class Material(models.Model):
    nombre = models.CharField(max_length=100)
    categoria = models.CharField(max_length=50)
    stock = models.PositiveIntegerField()
    precio = models.DecimalField(max_digits=10, decimal_places=2)
    descripcion = models.TextField(blank=True)

    def __str__(self):
        return self.nombre
```

4.2 Formularios y validación

`materiales/forms.py`

```

materiales/forms.py
from django import forms

from .models import Material

class MaterialForm(forms.ModelForm):
    class Meta:
        model = Material
        fields = ["nombre", "categoria", "stock", "precio", "descripcion"]
        widgets = {
            "nombre": forms.TextInput(attrs={"class": "form-control"}),
            "categoria": forms.TextInput(attrs={"class": "form-control"}),
            "stock": forms.NumberInput(attrs={"class": "form-control"}),
            "precio": forms.NumberInput(attrs={"class": "form-control", "step": "0.01"}),
            "descripcion": forms.Textarea(attrs={"class": "form-control", "rows": 4}),
        }

    def clean_precio(self):
        precio = self.cleaned_data.get("precio")
        if precio is not None and precio <= 0:
            raise forms.ValidationError("El precio debe ser mayor que cero.")
        return precio

    def clean_stock(self):
        stock = self.cleaned_data.get("stock")
        if stock is not None and stock < 0:
            raise forms.ValidationError("El stock no puede ser negativo.")
        return stock

```

4.3 Vistas del CRUD

materiales/views.py

```

materiales/views.py
from django.shortcuts import get_object_or_404, redirect, render

from .forms import MaterialForm
from .models import Material

def listar_materiales(request):
    materiales = Material.objects.all().order_by("nombre")
    return render(request, "materiales/lista.html", {"materiales": materiales})

def crear_material(request):
    if request.method == "POST":
        form = MaterialForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect("materiales:lista")
    else:
        form = MaterialForm()

    return render(request, "materiales/formulario.html", {"form": form, "accion": "Crear"})

def editar_material(request, pk):
    material = get_object_or_404(Material, pk=pk)

    if request.method == "POST":
        form = MaterialForm(request.POST, instance=material)

```

4.4 Rutas del sistema

materiales/urls.py

```

materiales/urls.py
from django.urls import path

from . import views

app_name = "materiales"

urlpatterns = [
    path("", views.listar_materiales, name="lista"),
    path("crear/", views.crear_material, name="crear"),
    path("editar/<int:pk>/", views.editar_material, name="editar"),
    path("eliminar/<int:pk>/", views.eliminar_material, name="eliminar"),
]

```

4.5 Templates

Se crearon las plantillas para listar, crear, editar y confirmar eliminación.

materiales/templates/materiales/lista.html

```
materiales/templates/materiales/lista.html
{% extends 'catalogo/base.html' %}

{% block title %}Materiales | Alacena & Molde{% endblock %}

{% block content %}
<main class="materials-page">
  <section class="materials-header">
    <div>
      <p class="eyebrow">MATERIALES</p>
      <h1>Inventario de <em>materiales</em></h1>
    </div>
    <a class="primary-button" href="{% url 'materiales:crear' %}">Agregar material <span>⌕</span></a>
  </section>

  {% if materiales %}
  <div class="materials-table-wrap">
    <table class="materials-table">
      <thead>
        <tr>
          <th>Nombre</th>
          <th>Categoría</th>
          <th>Stock</th>
          <th>Precio</th>
          <th>Descripción</th>
          <th>Acciones</th>
        </tr>
      </thead>
      <tbody>

```

materiales/templates/materiales/formulario.html

```
materiales/templates/materiales/formulario.html
{% extends 'catalogo/base.html' %}

{% block title %}{% accion %} material | Alacena & Molde{% endblock %}

{% block content %}
<main class="form-page">
  <section class="form-card">
    <p class="eyebrow">{% accion|upper %} MATERIAL</p>
    <h1>{% accion %} material</h1>

    <form method="post" novalidate>
      {% csrf_token %}
      {{ form.non_field_errors }}

      {% for field in form %}
      <div class="field-group">
        <label for="{% field.id_for_label %}">{% field.label %}</label>
        {{ field }}
        {% if field.errors %}
        <div class="error-text">{% field.errors|striptags %}</div>
        {% endif %}
      </div>
      {% endfor %}

      <div class="form-actions">
        <button class="primary-button" type="submit">Guardar</button>
        <a class="secondary-button" href="{% url 'materiales:lista' %}">Cancelar</a>
      </div>

```

materiales/templates/materiales/eliminar.html

```
materiales/templates/materiales/eliminar.html
{% extends 'catalogo/base.html' %}

{% block title %}Eliminar material | Alacena & Molde{% endblock %}

{% block content %}
<main class="form-page">
  <section class="form-card confirm-card">
    <p class="eyebrow">CONFIRMACIÓN</p>
    <h1>Eliminar material</h1>
    <p>¿Estás seguro de que deseas eliminar <strong>{% material.nombre %}</strong>?</p>

    <form method="post">
      {% csrf_token %}
      <div class="form-actions">
        <button class="primary-button danger-button" type="submit">Confirmar eliminación</button>
        <a class="secondary-button" href="{% url 'materiales:lista' %}">Cancelar</a>
      </div>
    </form>
  </section>
</main>
{% endblock %}

```

5. Pruebas realizadas

Se ejecutaron pruebas para comprobar el correcto funcionamiento del sistema. Se validaron los casos de listado, creación, edición, cancelación de eliminación, confirmación, manejo de 404, validación de errores y seguridad CSRF.

materiales/tests.py

```
materiales/tests.py
from django.contrib.auth import get_user_model
from django.test import Client, TestCase
from django.urls import reverse

from .models import Material

class MaterialCrudTests(TestCase):
    def setUp(self):
        self.material = Material.objects.create(
            nombre="Harina 0000",
            categoria="Ingredientes",
            stock=25,
            precio=150.00,
            descripcion="Harina para pan y repostería.",
        )
        self.list_url = reverse("materiales:lista")
        self.crear_url = reverse("materiales:crear")
        self.editar_url = reverse("materiales:editar", args=[self.material.pk])
        self.eliminar_url = reverse("materiales:eliminar", args=[self.material.pk])

    def test_listado_muestra_materiales(self):
        response = self.client.get(self.list_url)
        self.assertEqual(response.status_code, 200)
        self.assertContains(response, "Harina 0000")
        self.assertTemplateUsed(response, "materiales/lista.html")

    def test_materiales_disponibles_en_admin(self):
```

6. Resultados verificados

Verificación

```
Resultados reales de pruebas
$ python manage.py test materiales --verbosity=2
Found 10 test(s).
System check identified no issues (0 silenced).
test_campos_vacios (materiales.tests.MaterialCrudTests.test_campos_vacios) ... ok
test_cancelar_eliminacion_no BORRA registro (materiales.tests.MaterialCrudTests.test_cancelar_eliminacion_no BORRA registro) ... ok
test_confirmar_eliminacion (materiales.tests.MaterialCrudTests.test_confirmar_eliminacion) ... ok
test_creacion_valida (materiales.tests.MaterialCrudTests.test_creacion_valida) ... ok
test_csrf_requiere_token (materiales.tests.MaterialCrudTests.test_csrf_requiere_token) ... ok
test_dato_invalido (materiales.tests.MaterialCrudTests.test_dato_invalido) ... ok
test_editar_material (materiales.tests.MaterialCrudTests.test_editar_material) ... ok
test_error_404_para_registro_inexistente (materiales.tests.MaterialCrudTests.test_error_404_para_registro_inexistente) ... ok
test_listado_muestra_materiales (materiales.tests.MaterialCrudTests.test_listado_muestra_materiales) ... ok
test_materiales_disponibles_en_admin (materiales.tests.MaterialCrudTests.test_materiales_disponibles_en_admin) ... ok
Ran 10 tests in 0.289s
OK

$ python manage.py check
System check identified no issues (0 silenced).

Exit codes: tests=0, check=0
```

7. Capturas de pruebas web

Las siguientes capturas se tomaron del sitio local durante las pruebas. El registro creado para las capturas fue temporal y se eliminó al terminar.

Listado con materiales de ejemplo

MATERIALES

Inventario de
materiales

Agregar
material



NOMBRE	CATEGORÍA	STOCK	PRECIO	DESCRIPCIÓN
Cacao amargo	Ingredientes	14	\$3200,00	Cacao amargo para cobertura
Harina 0000	Ingredientes	25	\$1850,00	Harina 0000 para repostería y masa
Manteca de cacao	Ingredientes	18	\$280,00	Manteca de cacao para rellenos
Molde de silicona	Utensilios	8	\$4600,00	Molde de silicona para preparar

Formulario para crear

CREAR MATERIAL

Crear material

NOMBRE

CATEGORIA

STOCK

PRECIO

DESCRIPCION

Guardar

Cancelar

CREAR MATERIAL

Crear material

NOMBRE

Este campo es obligatorio.

CATEGORIA

Este campo es obligatorio.

STOCK

Este campo es obligatorio.

PRECIO

Este campo es obligatorio.

DESCRIPCION

Guardar

Cancelar

Rechazo de stock y precio inválidos

CREAR MATERIAL

Crear material

NOMBRE

Material inválido

CATEGORIA

Prueba

STOCK

-3

Asegúrese de que este valor es mayor o igual a 0.

PRECIO

0

El precio debe ser mayor que cero.

DESCRIPCION

No debe guardarse.

Guardar

Cancelar

Registro creado

MATERIALES

Inventario de *materiales*

Agregar
material +

NOMBRE	CATEGORÍA	STOCK	PRECIO	DESCRIPCIÓN
Cacao amargo	Ingredientes	14	\$3200,00	Cacao en polvo y cobertura
Captura temporal CRUD	Prueba	2	\$99,50	Registro de captura temporal
Harina 0000	Ingredientes	25	\$1850,00	Harina para repostería y bizcochos
Manteca de cacao	Ingredientes	18	\$280,00	Manteca para rellenar y decorar
Molde de silicona	Utensilios	8	\$4600,00	Molde para preparar postres

EDITAR MATERIAL

Editar material

NOMBRE

Captura temporal CRUD

CATEGORIA

Prueba

STOCK

2

PRECIO

99.50

DESCRIPCION

Registro temporal para las capturas.

Guardar

Cancelar

Edición guardada

MATERIALES

Inventario de materiales

Agregar material +

NOMBRE	CATEGORÍA	STOCK	PRECIO	DESCRIPCIÓN
Cacao amargo	Ingredientes	14	\$3200,00	Cacao y cobe
Captura temporal editada	Prueba	2	\$99,50	Registr
Harina 0000	Ingredientes	25	\$1850,00	Harina bizco
Manteca de cacao	Ingredientes	18	\$280,00	Mantec relleno
Molde de silicona	Utensilios	8	\$4600,00	Molde prepar



CONFIRMACIÓN

Eliminar material

¿Estás seguro de que deseas eliminar **Captura temporal editada?**

Confirmar eliminación

Cancelar

Cancelación: el registro permanece

MATERIALES

Inventario de *materiales*

Agregar
material +

NOMBRE	CATEGORÍA	STOCK	PRECIO	DESCRIPCIÓN
Cacao amargo	Ingredientes	14	\$3200,00	Cacao y cobe
Captura temporal editada	Prueba	2	\$99,50	Registr
Harina 0000	Ingredientes	25	\$1850,00	Harina bizco
Manteca de cacao	Ingredientes	18	\$280,00	Mantec
Molde de silicona	Utensilios	8	\$4600,00	Molde prepar

Eliminación confirmada: el registro ya no aparece

MATERIALES

Inventario de *materiales*

Agregar
material +

NOMBRE	CATEGORÍA	STOCK	PRECIO	DESCRIPCIÓN
Cacao amargo	Ingredientes	14	\$3200,00	Cacao en cobertura
Harina 0000	Ingredientes	25	\$1850,00	Harina ref y masas li
Manteca de cacao	Ingredientes	18	\$280,00	Manteca p rellenos.
Molde de silicona	Utensilios	8	\$4600,00	Molde reu preparaci

Page not found (404)

No Material matches the given query.

Request Method: GET

Request URL: http://localhost:8000/materiales/editar/9999999/

Raised by: materiales.views.editar_material

Using the URLconf defined in `config.urls`, Django tried these URL patterns, in this order:

1. admin/
2. [name='inicio']
3. nosotros/ [name='nosotros']
4. inicio/
5. materiales/ [name='lista']
6. materiales/ crear/ [name='crear']
7. materiales/ editar/<int:pk>/ [name='editar']

The current path, `materiales/editar/9999999/`, matched the last one.

You're seeing this error because you have `DEBUG = True` in your Django settings file. Change that to `False`, and Django will display a standard 404 page.

403 al enviar el formulario sin token CSRF

Prohibido (403)

La verificación CSRF ha fallado. Solicitud abortada.

Help

Reason given for failure:

`CSRF token missing.`

In general, this can occur when there is a genuine Cross Site Request Forgery, or when [Django's CSRF mechanism](#) has not been used correctly. For POST forms, you need to ensure:

- Your browser is accepting cookies.
- The view function passes a `request` to the template's [render](#) method.
- In the template, there is a `{% csrf_token %}` template tag inside each POST form that targets an internal URL.
- If you are not using `CsrfViewMiddleware`, then you must use `csrf_protect` on any views that use the `csrf_token` template tag, as well as those that accept the POST data.
- The form has a valid CSRF token. After logging in in another browser tab or hitting the back button after a login, you may need to reload the page with the form, because the token is rotated after a login.

You're seeing the help section of this page because you have `DEBUG = True` in your Django settings file. Change that to `False`, and only the initial error message will be displayed.

You can customize this page using the `CSRF_FAILURE_VIEW` setting.

Administración de Django

Nombre de usuario:

Contraseña:

INICIAR SESIÓN

8. GitHub, uso de IA y reflexión

La rama publicada es entrega-materiales. El historial de Git debe revisarse para identificar las contribuciones reales de cada integrante; no se atribuyen commits individuales en este informe.

Prompt del equipo: “Trabajar principalmente en el CRUD web de materiales; reemplazar los materiales escritos manualmente por consultas ORM; crear funciones para listar, crear, editar y eliminar; usar `get_object_or_404()`; crear las rutas y plantillas; incorporar CSRF; ejecutar el borrado solo por POST; mantener la cancelación; probar listado, creación válida, campos vacíos, dato inválido, edición, cancelación, confirmación, 404 y CSRF; sacar capturas de las pruebas”.

Propuesta aplicada: ModelForm, operaciones ORM, rutas con nombres, validación, confirmación POST y pruebas automatizadas. **Decisión:** aceptar los cambios después de revisar archivos y

ejecutar pruebas; mantener PostgreSQL como requisito pendiente cuando no hay servidor local disponible. **Verificación:** diez pruebas automatizadas y manage.py check sin errores en SQLite; captura real de 403, 404 y flujos CRUD.

Reflexión del equipo (completar con sus propias respuestas):

¿Qué parte del CRUD fue más difícil y por qué?

¿Qué error permitió aprender algo importante?

¿Qué sugerencia de IA aceptaron, modificaron o rechazaron y por qué?

¿Qué funcionalidad implementarán en la siguiente evaluación?

Pendiente antes de entregar: configurar y demostrar PostgreSQL; ejecutar migrate y mostrar pgAdmin; crear superusuario y realizar las operaciones requeridas desde Admin; incorporar capturas de esas acciones; completar la reflexión y confirmar la distribución real de commits.

9. Conclusión

El CRUD web y sus validaciones están implementados y verificados localmente. La configuración admite PostgreSQL y Django Admin; la evaluación completa depende de comprobar la conexión PostgreSQL y adjuntar la evidencia administrativa pendiente indicada arriba.